CS 214 Spring 2026
Project II

David Menendez

Due: March 30 at 11:59 PM

For this assignment, you will write a program `compare` that analyzes a set of files and reports their similarity using by computing the Jenson-Shannon distance (JSD) for each pair of files in the set.

1 Submission

You will submit a Tar archive containing a directory `P2`, within which are:

1. Your complete source code for `compare`
2. A Makefile that can be used to compile your project
3. A README containing the names and NetIDs of both partners, along with a description of your testing plan and any design notes
4. An AUTHOR file containing only the NetIDs of both partners.

It is not necessary to include your test files. Do not include any compiled executables or object files.

We should be able to run the following commands as written (aside from the name of the archive and test directory):

```
$ tar -xf p2.tar
$ cd P2
$ make
$ ./compare path/to/test/directory
```

2 User interface

Your program will be given the names of one or more files and directories. The file names and directories are used to determine the set of files to analyze. File names given as arguments are added directly to the set of files. For each directory, your program will add any files whose names have a particular suffix to the set, and recursively traverse any subdirectories.

For each (unordered) pair of files in the analysis set, your program will output the Jensen-Shannon distance (JSD) between their word frequencies.

For example:

```
$ ls bar
baz.txt quux spam/
$ ls bar/spam
eggs.txt bacon.c
$ ./compare foo bar
0.03000 foo bar/baz.txt
0.12000 foo bar/spam/eggs.txt
0.80000 bar/baz.txt bar/spam/eggs.txt
```

In this example, the arguments are `foo` (a file) and `bar` (a directory). One file in `bar` matches the suffix `".txt"` and is added to the file set. The subdirectory `bar/spam` is also traversed, and its files that end with the suffix are added. This results in the file set `foo`, `bar/baz.txt`, and `bar/spam/eggs.txt`. (Note that `foo` does not need to end with the suffix, because it was explicitly given by the user.)

For each pair of files in the file set, the program outputs the JSD and the (path) names of the files being compared. Comparisons are printed in decreasing order of combined word count (that is, the total number of words in both files). Note that pairs are unordered, so the output will include comparisons of `foo` and `bar/baz.txt` or `bar/baz.txt` and `foo`, but not both.

2.1 Arguments

Your program will take one or more arguments, each of which must be the name of (or a path to) a file or directory.

Each file name is added to the set of files to examine.

Each directory name is added to the set of directories to traverse. For each directory in the set, your program will read through its entries. Any file whose name ends with the file name suffix is added to the set of files to examine. Any directory is added to the set of directories to traverse. Entries that are not directories and that do not end with the specified suffix are ignored.

Exception Any file or directory whose name begins with a period is skipped.

3 Operation

Your program will compute the word frequency distribution for each file in a set of files. For each pair of files in this set, it will compute the Jensen-Shannon distance (JSD) between the distributions for those files

3.1 Word frequency distribution (WFD)

The *frequency* of a word is the number of times the word appears divided by the total number of words. For example, in the text "spam eggs bacon spam", the frequency of "spam" is 0.5, and the frequencies of "eggs" and "bacon" are both 0.25.

The *word frequency distribution* (WFD) for a file is a list of every word that occurs at least once in the file, along with its frequency. Ignoring rounding errors, the frequencies for all the words will add up to 1.

Table 1: Word frequency distribution example

Text: `I can't understand thieves' cant.`

Word	Occurrences	Frequency
cant	2	0.4
i	1	0.2
thieves	1	0.2
understand	1	0.2

Tokenizing To compute the WFD for a file, your program must read the file and determine what words it contains. For this project, we define a word to be a sequence of word characters, where word characters include letters, numbers, and the dash (hyphen). Words are separated by whitespace characters. Other characters, such as punctuation, are ignored.

To compute the WFD, keep a list of every word found in the file and its count. Each time a word is encountered, look for it in the list and increment its count or add it with count 1. Once every word has been read, divide the count for each word by the total number of words.

Words are case-insensitive. The simplest way to handle this is to convert words to upper- or lowercase while reading.

Table 1 shows the WFD for a short example file. Note that "I" has been converted to lowercase and that "can't" and "cant" are treated as the same word (because the apostrophe is ignored).

Data structure You will need to maintain a list of mappings between words and counts (later frequencies). You are free to choose any data structure, but you may find a linked list to be a good balance of efficiency and simplicity. It is recommended that you choose a structure that allows you to iterate through the words in lexicographic order.

Your program must not assume a maximum word length. Instead, word storage will be dynamically allocated.

3.2 Jensen-Shannon distance (JSD)

The Jensen-Shannon distance (JSD) is a symmetric measure of the similarity of two discrete distributions. We will write $F_i(w)$ to mean the frequency of word w in file i. Note that $F_i(w) = 0$ if w does not appear in file i.

We define the *mean frequency* for a word to be the average of its frequencies in the two files.

$$\overline{F}(w) = \tfrac{1}{2}(F_1(w) + F_2(w)) \tag{1}$$

Next we compute the Kullbeck-Leibler Divergence (KLD) between each file and the mean.

$$KLD(F_i||\overline{F}) = \sum_w F_i(w) \cdot \log_2 \left(\frac{F_i(w)}{\overline{F}(w)} \right) \tag{2}$$

This is computed by adding the frequencies of each word in the file scaled by the base-2 logarithm of the frequency divided by the mean frequency for that word. The result will be a value between 0 and 1.

Finally, the JSD is the square root of the mean KLD between the files and the mean distribution.

$$JSD(F_1||F_2) = \sqrt{\tfrac{1}{2}KLD(F_1||\overline{F}) + \tfrac{1}{2}KLD(F_2||\overline{F})} \tag{3}$$

(a) F_1

hi	0.5
there	0.5

(b) F_2

hi	0.5
out	0.25
there	0.25

(c) $\overline{F}$

hi	0.5
out	0.125
there	0.375

$$
\begin{aligned}
KLD(F_1||\overline{F}) &= 0.5 \cdot \log_2\left(\frac{0.5}{0.5}\right) + 0.5 \cdot \log_2\left(\frac{0.5}{0.375}\right) \\
&\approx 0.5 \cdot 0 + 0.5 \cdot 0.415 \\
&\approx 0.2075 \\
KLD(F_2||\overline{F}) &= 0.5 \cdot \log_2\left(\frac{0.5}{0.5}\right) + 0.25 \cdot \log_2\left(\frac{0.25}{0.125}\right) + 0.25 \cdot \log_2\left(\frac{0.25}{0.375}\right) \\
&\approx 0.5 \cdot 0 + 0.25 \cdot 1 + 0.25 \cdot -0.585 \\
&\approx 0.1038 \\
JSD(F_1||F_2) &\approx \sqrt{\tfrac{1}{2}0.2075 + \tfrac{1}{2}0.1038} \\
&\approx 0.3945
\end{aligned}
$$

Figure 1: Computing the JSD for two files

Figure 1 shows the steps to compute the JSD for two files containing "hi there hi there" and "hi hi out there", respectively. Figures 1a and 1b give the WFD for the files, and fig. 1c shows their mean WFD.

Implementation If you have chosen a WFD structure that allows you to iterate through the word lists in alphabetical order, it is simple to compute the JSD using simultaneous iteration for both lists. Keep a running total of the KLD for both files. If you encounter a word appearing in both lists, compute the mean frequency and then add the scaled frequencies to both running totals. If word appears in one list but not the other, treat its frequency as 0 in the file where it does not appear. Once all words have been considered, compute the JSD.

You must use the Posix functions `open()`, `read()`, and `close()` for reading files.

Using `sqrt()` and `log2()` is permitted, but be aware that you must use `-lm` to tell GCC to include the math library when linking.

4 Program organization

Your program will operate in two phases. The collection phase recursively traverses directories and reads files to find the WFD (see section 3.1) for each requested file. The analysis phase computes the JSD (see section 3.2) for each pair of requested files.

Error conditions If any file or directory cannot be opened, report an error and continue processing. It is sufficient to call `perror()` with the name of the file or directory to report the error. You may also exit with status `EXIT_FAILURE` when the program completes.

For unexpected errors, such as `malloc()` returning `NULL`, your program may terminate immediately.

4.1 Analysis phase

The analysis phase computes the JSD (see section 3.2) and combined word count for each pair of files.

Data structure For each comparison, we will need the names of the files being compared, the combined word count, and the JSD. If there are n files, there will be $\frac{1}{2}n(n+1)$ comparisons, so it is feasible to create an array of structs, compute the results for each pairing, and finally sort the array using `qsort()`.

Error conditions If the collection phase found fewer than two files, report an error and exit.

Any numeric errors occurring in this phase (e.g., word frequencies outside the range [0,1]) indicate problems in the collection phase. You may choose to check these using `assert()`.

5 Advice

Don't panic! While this assignment has many components, the components themselves are relatively simple.

During debugging, you may find it helpful to print log messages describing what actions your threads are doing. Use the `DEBUG` macro trick to easily enable or disable these messages as needed.

Always check for errors. Functions like `malloc()` or `pthread_mutex_lock()` never fail under normal circumstances, which means you definitely want to be informed if they do fail—it may indicate a logic error in your program! If writing the error checks becomes tedious, define a macro to write them for you. How best to handle error conditions is a design choice, but the usual thing to do when something "impossible" happens is to report an error and terminate the program.

That being said, checking `printf()` for an error result is probably overkill.

6 Grading

Your program will be scored out of 100 points. Scoring will be based on your README (especially your test plan) and how well your code handles our test scenarios. We may additionally add or remove points based on the quality and readability of your code. Testing will be performed on iLab machines, so be certain that your code will compile and execute on the iLab!

Your code should be free of memory errors and undefined behavior. We may compile your code using AddressSanitizer, UBSan, Valgrind, or other analysis tools. You will lose points if these tools report memory errors, space leaks, or undefined behavior. You are advised to take advantage of AddressSanitizer and UBSan when compiling your program for testing.